

WEEK 10

Aim:

To develop a Student Registration web application using the MERN stack (MongoDB, Express.js, React.js, Node.js) that allows users to register student details and perform Create, Read, Update and Delete (CRUD) operations on a MongoDB database using Mongoose and the Fetch API.

Description:

This experiment demonstrates a complete full-stack CRUD application. The React frontend uses the `useState()` and `useEffect()` hooks to manage form data and to load records when the page opens. The form collects the student's name, email, password, gender (radio buttons), country (dropdown) and languages (checkboxes). When the Register button is clicked, React sends the data to the Express.js server using `fetch()`. The server stores it in MongoDB through a Mongoose schema and model. The stored records are retrieved and displayed in a table below the form, and each record has Edit and Delete buttons.

The CRUD workflow is: React UI → `fetch()` → Express.js → Mongoose → MongoDB → Express.js → React UI.

Technologies Used

- Frontend: React.js (Vite)
- Backend: Node.js + Express.js
- Database: MongoDB (database: studentdb, collection: students)
- Database library: Mongoose
- Communication: Fetch API

Project Structure

```
week10-mern-student-registration/  
  server/  
    server.js  
    package.json  
  client/  
    index.html  
    package.json  
    vite.config.js  
    src/  
      main.jsx  
      App.jsx  
      App.css
```

Program:

Student Registration CRUD Application Using the MERN Stack

Program Code :

server/server.js

```
import express from "express";  
import mongoose from "mongoose";  
import cors from "cors";  
  
const app = express();  
  
app.use(cors());  
app.use(express.json());
```

```

// MongoDB connection
mongoose
  .connect("mongodb://127.0.0.1:27017/studentdb")
  .then(() => {
    console.log("MongoDB connected");
  })
  .catch((error) => {
    console.log("MongoDB connection error:", error.message);
  });

// Schema means structure of data to be stored in database
const studentSchema = new mongoose.Schema({
  name: String,
  email: String,
  password: String,
  gender: String,
  country: String,
  languages: [String]
});

// Model (creates the "students" collection)
const Student = mongoose.model("Student", studentSchema);

// Test route
app.get("/", (req, res) => {
  res.send("Express server is working");
});

// READ
app.get("/students", async (req, res) => {
  const students = await Student.find();
  res.json(students);
});

// CREATE
app.post("/students", async (req, res) => {
  const student = new Student(req.body);
  await student.save();
  res.json({ message: "Registration successful" });
});

// UPDATE
app.put("/students/:id", async (req, res) => {
  await Student.findByIdAndUpdate(req.params.id, req.body);
  res.json({ message: "Student updated successfully" });
});

// DELETE
app.delete("/students/:id", async (req, res) => {
  await Student.findByIdAndDelete(req.params.id);
  res.json({ message: "Student deleted successfully" });
});

// Start server
app.listen(5000, () => {
  console.log("Server running on port 5000");
});

```

client/src/main.jsx

```

import { StrictMode } from "react";
import { createRoot } from "react-dom/client";
import App from "../App.jsx";

createRoot(document.getElementById("root")).render(
  <StrictMode>
    <App />
  </StrictMode>
);

```

client/src/App.jsx

```

import './App.css'

import { useEffect, useState } from "react";

```

```

const API = "http://localhost:5000";

function App() {

  const emptyForm = {
    name: "",
    email: "",
    password: "",
    gender: "",
    country: "",
    languages: []
  };

  const [form, setForm] = useState(emptyForm);

  const [students, setStudents] = useState([]);

  const [message, setMessage] = useState("");

  const [editId, setEditId] = useState("");

  // Get students when page opens
  useEffect(() => {
    loadStudents();
  }, []);

  // READ
  function loadStudents() {

    fetch(API + "/students")
      .then(response => response.json())
      .then(data => {
        setStudents(data);
      })
      .catch(error => {
        console.log(error);
        setMessage("Start the Express server first");
      });
  }

  // Text, email, password, radio and select
  function handleChange(event) {

    const { name, value } = event.target;

    setForm({
      ...form,
      [name]: value
    });
  }

  // Checkbox
  function handleLanguage(event) {

    const value = event.target.value;
    const checked = event.target.checked;

    let languages = form.languages;

    if (checked) {
      languages = [...languages, value];
    } else {
      languages = languages.filter(
        item => item !== value
      );
    }

    setForm({
      ...form,

```

```

        languages: languages
    });
}

// CREATE or UPDATE
function saveStudent(event) {

    event.preventDefault();

    let url = API + "/students";

    let method = "POST";

    if (editId !== "") {

        url = API + "/students/" + editId;

        method = "PUT";
    }

    fetch(url, {
        method: method,

        headers: {
            "Content-Type": "application/json"
        },

        body: JSON.stringify(form)
    })
    .then(response => response.json())
    .then(data => {

        setMessage(data.message);

        setForm(emptyForm);

        setEditId("");

        loadStudents();

    })
    .catch(error => {

        console.log(error);

        setMessage("Could not connect to Express");

    });
}

// EDIT
function editStudent(student) {

    setForm({
        name: student.name,
        email: student.email,
        password: student.password,
        gender: student.gender,
        country: student.country,
        languages: student.languages
    });

    setEditId(student._id);

    setMessage("Edit the data and click Update");
}

// DELETE

```

```

function deleteStudent(id) {

  fetch(API + "/students/" + id, {
    method: "DELETE"
  })
    .then(response => response.json())
    .then(data => {

      setMessage(data.message);

      loadStudents();

    })
    .catch(error => {

      console.log(error);

      setMessage("Could not connect to Express");

    });
}

```

```

// RESET
function resetForm() {

  setForm(emptyForm);

  setEditId("");

  setMessage("");
}

```

```

return (

  <div className="page">

    <div className="form-box">

      <h1>Student Registration</h1>

      <form onSubmit={saveStudent}>

        <label>Name</label>

        <input
          name="name"
          value={form.name}
          onChange={handleChange}
          required
        />

        <label>Email</label>

        <input
          type="email"
          name="email"
          value={form.email}
          onChange={handleChange}
          required
        />

        <label>Password</label>

        <input
          type="password"

```

```

        name="password"
        value={form.password}
        onChange={handleChange}
        required
    />

<label>Gender</label>

<div className="row">

    <label>
        <input
            type="radio"
            name="gender"
            value="Male"
            checked={form.gender === "Male"}
            onChange={handleChange}
        />
        Male
    </label>

    <label>
        <input
            type="radio"
            name="gender"
            value="Female"
            checked={form.gender === "Female"}
            onChange={handleChange}
        />
        Female
    </label>

</div>

<label>Country</label>

<select
    name="country"
    value={form.country}
    onChange={handleChange}
    required
>

    <option value="">
        Select Country
    </option>

    <option value="India">
        India
    </option>

    <option value="USA">
        USA
    </option>

    <option value="UK">
        UK
    </option>

    <option value="Australia">
        Australia
    </option>

</select>

<label>Languages</label>

<div className="row">

```

```
<label>
  <input
    type="checkbox"
    value="English"
    checked={form.languages.includes("English")}
    onChange={handleLanguage}
  />
  English
</label>
```

```
<label>
  <input
    type="checkbox"
    value="Hindi"
    checked={form.languages.includes("Hindi")}
    onChange={handleLanguage}
  />
  Hindi
</label>
```

```
<label>
  <input
    type="checkbox"
    value="Telugu"
    checked={form.languages.includes("Telugu")}
    onChange={handleLanguage}
  />
  Telugu
</label>
```

```
</div>
```

```
<button type="submit">
  {editId === "" ? "Register" : "Update"}
</button>
```

```
<button
  type="button"
  onClick={resetForm}
>
  Reset
</button>
```

```
</form>
```

```
{message && (
  <div className="message">
    {message}
  </div>
)}
```

```
</div>
```

```
<div className="data-box">
```

```
<h2>Registered Students</h2>
```

```
<table>
```

```
<thead>
```

```
<tr>
  <th>Name</th>
  <th>Email</th>
  <th>Gender</th>
  <th>Country</th>
```

```

        <th>Languages</th>
        <th>Actions</th>
      </tr>
    </thead>

    <tbody>

      {students.map(student => (

        <tr key={student._id}>

          <td>{student.name}</td>

          <td>{student.email}</td>

          <td>{student.gender}</td>

          <td>{student.country}</td>

          <td>
            {student.languages.join(", ")}
          </td>

          <td>

            <button
              onClick={() => editStudent(student)}
            >
              Edit
            </button>

            <button
              onClick={() =>
                deleteStudent(student._id)
              }
            >
              Delete
            </button>

          </td>

        </tr>

      )})

    </tbody>

  </table>

</div>

</div>
);
}

```

```
export default App;
```

client/src/App.css

```

* { box-sizing: border-box; }

body {
  margin: 0;
  font-family: Arial, sans-serif;
  background: #f1f5f9;
  color: #1e293b;
}

```

```

.page {
  max-width: 900px;
  margin: 0 auto;
  padding: 24px 16px;
  display: flex;
  flex-direction: column;
  gap: 24px;
}

.form-box, .data-box {
  background: #fff;
  padding: 24px;
  border-radius: 10px;
  box-shadow: 0 2px 8px rgba(0, 0, 0, 0.08);
}

h1, h2 { margin-top: 0; }

form { display: flex; flex-direction: column; gap: 8px; }

label { font-weight: bold; font-size: 14px; }

input:not([type="radio"]):not([type="checkbox"]), select {
  padding: 8px;
  border: 1px solid #cbd5e1;
  border-radius: 6px;
  font-size: 14px;
}

.row { display: flex; gap: 20px; flex-wrap: wrap; }
.row label { font-weight: normal; display: flex; align-items: center; gap: 4px; }

button {
  padding: 8px 16px;
  margin: 8px 8px 0 0;
  border: none;
  border-radius: 6px;
  background: #2563eb;
  color: #fff;
  cursor: pointer;
  font-size: 14px;
}
button:hover { background: #1d4ed8; }
button[type="button"] { background: #64748b; }

.message {
  margin-top: 12px;
  padding: 10px;
  background: #dcfce7;
  border-radius: 6px;
}

table { width: 100%; border-collapse: collapse; }
th, td { padding: 8px; border-bottom: 1px solid #e2e8f0; text-align: left; font-size: 14px; }
th { background: #f8fafc; }
td button { margin: 0 4px 0 0; padding: 4px 10px; }

```

Data : MongoDB Document

Database: studentdb Collection: students. Each student document contains the following fields:

Field	Type	Form Control
name	String	Text box
email	String	Email input
password	String	Password input
gender	String	Radio buttons
country	String	Select / Dropdown
languages	[String]	Checkboxes

Execution Of the program

REST API routes

Method	URL / Route	Purpose
GET	http://localhost:5000/	Test route, confirms Express server is working
GET	http://localhost:5000/students	Retrieves all students from MongoDB
POST	http://localhost:5000/students	Registers a new student
PUT	http://localhost:5000/students/:id	Updates a student
DELETE	http://localhost:5000/students/:id	Deletes a student

Steps to run

1. Make sure the MongoDB service is running.
2. Server: open a terminal, go to the server folder and run: `npm install` then `npm start`. The terminal shows "Server running on port 5000" and "MongoDB connected".
3. Client: open a second terminal, go to the client folder and run: `npm install` then `npm run dev`.
4. Open `http://localhost:5173` in the browser.

Output:

1) Create & Read : Registration form with success message and table : localhost:5173

Student Registration

Name

Email

Password

Gender
☐ Male ☐ Female

Country

Languages
☐ English ☐ Hindi ☐ Telugu

Registration successful

Registered Students

Name	Email	Gender	Country	Languages	Actions
tharun	brisha@gmail.com	Male	India	English, Hindi, Telugu	Edit Delete

Form filled, Register clicked, showing "Registration successful" and the registered students in the table.

2) Update : after clicking Edit and Update

Student Registration

Name

Email

Password

Gender
☒ Male ☐ Female

Country

Languages
☒ English ☐ Hindi ☐ Telugu

Edit the data and click Update

Registered Students

Name	Email	Gender	Country	Languages	Actions
pranaay	andalurishanmukharao.24.csm@anits.edu.in	Male	USA	English	Edit Delete

3) Delete : record remove

Student Registration

Name

Email

Password

Gender
☐ Male ☐ Female

Country

Languages
☐ English ☐ Hindi ☐ Telugu

Student deleted successfully

Registered Students

Name	Email	Gender	Country	Languages	Actions
pranaay	andalurishanmukharao.24.csm@anits.edu.in	Male	USA	English	Edit Delete

Message "Student deleted successfully" and the table refreshed.

Result:

The Student Registration CRUD application was successfully developed using the MERN stack. The application demonstrates React form handling with `useState()` and `useEffect()`, Fetch API communication, Express.js REST API routes (GET, POST, PUT, DELETE), Mongoose schema and model, and Create, Read, Update and Delete operations on the studentdb database in MongoDB.